Storiiboard 2.0 — Program Plan & Claude Code Hand-off

Oct 31, 2026 · @Kevin Matos

Vision and positioning

Mission: give people in film and content production the tools for their projects without depending on a handful of corporations and subscription sites. The tool is theirs: it runs on their machine, their projects stay in their own folders, and the code is open for them to change.

Storiiboard 2.0 makes storyboarding fast, and every frame connects straight to production: each shot knows its scene, location and shoot day, so schedules, shot lists and call sheets build themselves from the boards. References and decks support the storyboard; they are not the center.

Canva and Milanote are general tools. Neither knows what a shot, a lens or a location is, so filmmakers rebuild the same information three times: moodboard in Milanote, deck in Canva or Keynote, shot list in a spreadsheet. That gap is the wedge.

- **Against Milanote:** references on the board become structured frames with shot metadata, not loose images.
- **Against Canva:** decks and treatments are generated from the frames and board, with film-native templates (treatment, lookbook, director's pitch, one-pager).
- **Against both:** the same data exports as a shot list and feeds call sheets, so pre-production has one source of truth.

Against StudioBinder and spreadsheets, call sheets are built per shoot day from the project itself, with weather, sunrise, golden hour and maps pulled in automatically.

Who and how: built for filmmakers, producers and creators. Kevin uses it first on real jobs (Bey County is the natural pilot), then releases it as a free, open-source desktop app for Mac and Windows. Projects are ordinary folders on the user's own drive, saved automatically, with no account or server. Anyone can run it, fork it and improve it; Patreon funds the work.

Where the repo stands

The current mtsm-ph/storiiboard repo is a strong clickable design prototype, not an app: keep it as the visual spec and rebuild on a real foundation. Last commit was April 27, 2026.

Area	Finding	Implication
Size	About 3,100 lines: index.html plus 7 JSX files	Small enough to port screen by screen
Build	React 18 and Babel loaded from a CDN, compiled in the browser; no package.json	Needs a real Vite + TypeScript scaffold
Data	Every screen runs on hard-coded mock data in component state	Nothing saves; refresh wipes everything
Backend	No Supabase, auth or storage yet, despite the original spec	Whole data layer is new work
Canvas	No Konva or canvas engine; boards are styled divs	Board and deck editor are new work
Screens	Projects, storyboard frames, pitch mode, pitch deck generator, moodboard, assets, crew, calendar, notifications, history, producer panel, settings	Good UI reference for Phase 1 to 4
Access	Password gate checks a password written in the page's own JavaScript	Anyone who views source can read it; treat it as no protection

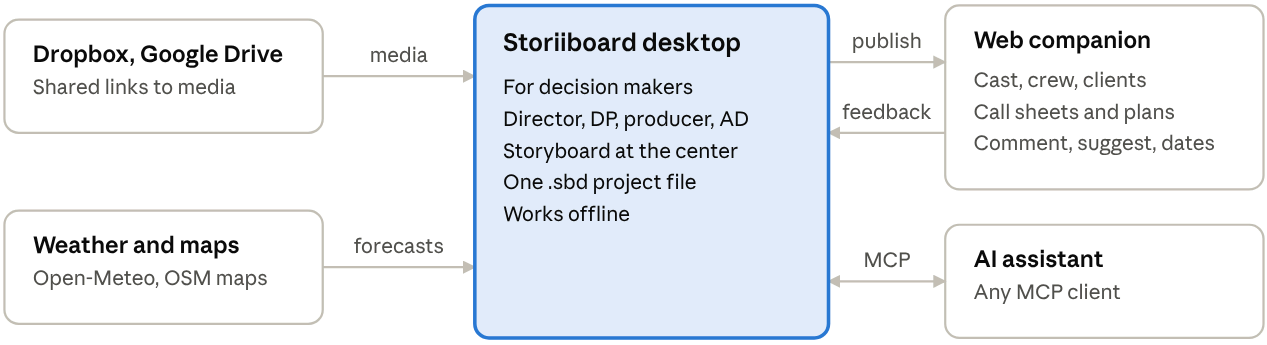
First fix in Phase 0: remove the client-side password and change it anywhere else it is reused. A local desktop app needs no login at all, so the gate simply goes away.

Product scope

Storiiboard has three parts: a desktop app where production decisions are made, a web companion where everyone else sees what they need, and an AI assistant that connects through MCP. Every tool below can be built; the notes column says where it needs extra care.

One desktop file feeds the web companion and the AI assistant

The storyboard is the core; everything else reads from the same .sbd project



surfaces · desktop app, web companion, AI assistant

The desktop app owns the project. The web gets a compressed snapshot and sends feedback back for approval.

Desktop tools

Tool	What it does	Main users	Can we build it
Storyboard (core)	Scenes and frames; each frame takes an image, photo, sketch or video still; shot size, lens, movement, INT/EXT, action, dialogue; auto shot numbers; keyboard-first	Director, DP	Yes
Video player and frame capture	Play footage and references frame by frame; set in and out; crop; grab a still or short clip straight into a frame	Director, DP, editor	Yes. Footage gets a light playback copy, because Mac and Windows play different formats natively
Image attachments	Attach images, PDFs and links to any shot, scene, location or person	Everyone	Yes

Tool	What it does	Main users	Can we build it
Bird's-eye view	Top-down plan per setup: walls, doors, windows, set pieces, cast marks and movement, cameras with true field of view from lens and sensor, lights with type and beam, flags and diffusion; a layer per department	DP, gaffer, key grip, art department	Yes. 2D first; it is the largest new tool
Locations	Address lookup, map, photos, contacts, parking and power notes	Producer, AD, location scout	Yes
Cast and crew	People, departments, roles, characters, contacts	Producer, AD	Yes
Gear lists	Camera, lighting, grip and sound packages per day, linked to the shots that need them	DP, gaffer, sound	Yes
Schedule	Shots grouped into shoot days by location and setup	AD, producer	Yes
Call sheets	Document built per day that links cast, scenes, locations, gear, crew by department, sound, weather, sun times, map and nearest hospital; PDF and web	AD, producer	Yes

Tool	What it does	Main users	Can we build it
Moodboards by department	Boards for creative direction, set design, makeup and wardrobe, linked to scenes and characters	Creative director, production designer, HMU, wardrobe	Yes
Pitch decks	Decks and treatments built from frames, with film templates	Director, producer	Yes
Cloud links	Paste a Dropbox or Google Drive shared link per project; the app lists the files, downloads them and relinks media	Everyone	Yes for public links; private folders need sign-in later
Find missing files	Search the computer for media that moved or was never linked, matched by name, size and length	Everyone	Yes, later phase
AI assistant	Reads the whole project, checks for mistakes, drafts changes for approval	Producer, AD, director	Yes, through MCP

Script and 3D tools

The 3D set stays light inside Storiiboard: simple blocks you can look through each camera at. Anyone who wants more opens the scene in Blender on their own computer, so heavy 3D work never runs on a server or on machines that can't handle it.

Tool	What it does	Main users	Can we build it
Script import	Read a Fountain, Final Draft (.fdx) or PDF script and create a shell for every scene: number, INT/EXT, location, DAY/NIGHT, speaking characters, estimated page length. Locations and cast roles are created along the way	Director, AD, producer	Yes. Scene headings follow fixed rules (Fountain syntax), so this works without AI
Script breakdown	The AI assistant reads each scene and tags props, wardrobe, vehicles, stunts and VFX, as suggestions to approve	AD, producer, departments	Yes, through MCP
3D set preview	Turns the bird's-eye plan into simple 3D: walls raised to height, cast as plain figures, lights as icons with beam cones. Look through any camera at its real field of view	Director, DP, gaffer	Yes. A light blockout runs on an ordinary laptop
Open in Blender	Export the set as glTF (an open 3D format with cameras and lights), then build a .blend with a camera per setup, real light types and a collection per department, using the Blender already on that person's computer	DP, gaffer, art department	Yes

Tool	What it does	Main users	Can we build it
Simple Blender layout	A Storiiboard startup layout for Blender that hides most of the interface and keeps the few tools a DP needs: move, rotate, camera view, lens, lights	Anyone new to Blender	Yes, through Blender's application templates (manual)
AI in Blender	An existing community MCP server lets Claude control Blender directly (blender-mcp). With Storiiboard's MCP it can build or adjust the scene from the plan	Advanced users	Yes, optional. It can run any code inside Blender, so save first

Web companion

Feature	What people can do
Call sheet	See the latest call sheet for their days, with their call time and scenes
Storyboard viewer	Browse compressed frames and shot info; no video, no editing tools
Bird's-eye viewer	See the latest plan for each scene, read-only
Availability	The producer proposes dates for a scene; each person marks the dates they can make. Whole days only, no hours
Comments and suggestions	Leave feedback on any frame, plan or call sheet, or suggest a change; it lands in the desktop app for approval
Company workspace	A brand groups its people: CEO and marketing approve, creators plan and shoot

The web companion is the only part that needs a server, so it has a running cost and holds private data such as phone numbers and addresses. Each person gets a private invite link, contact details show only to people on that call sheet, and snapshots expire after the shoot. Supabase's free tier covers a small production, and open-source users can host their own.

Who uses what

Role	Where they work	What they do
Director	Desktop	Everything; approves suggestions
DP	Desktop	Storyboard, camera layer of the bird's-eye view, 3D set, camera gear
Gaffer, key grip	Desktop or web	Lighting and grip layers, their gear lists; suggest changes
Producer, AD	Desktop	Script import, schedule, call sheets, availability, cast and crew
Creative director, production designer, HMU, wardrobe	Desktop or web	Their department moodboards; suggest changes
Sound	Web	Call sheet, sound notes and gear; suggest changes
Editor	Web or desktop	Storyboard and shot list with scene and shot numbers that match the slate; boards as PDF; marker export for Resolve and Premiere later
Cast	Web	Their call sheet, their scenes, availability
Client, marketing, CEO	Web	Storyboard, decks, approvals, comments
Content creators	Desktop	Their own projects inside a company workspace

Cut line: desktop tools ship before the web companion. A tool has to work on a real shoot before anyone else sees it.

From Kevin's Notion

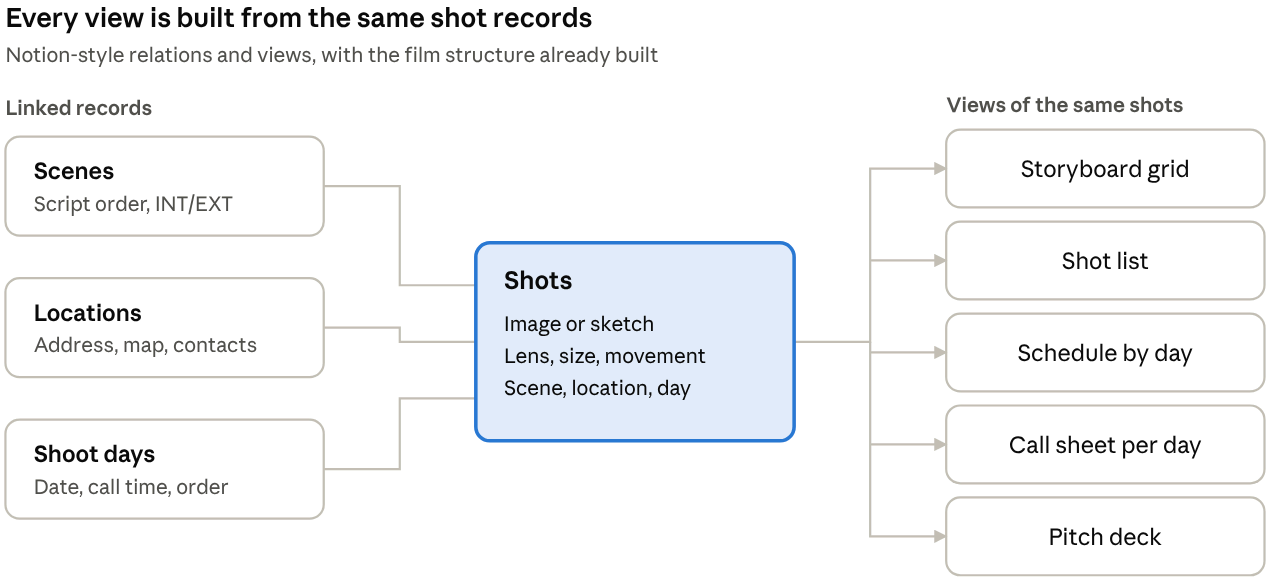
Kevin's existing Notion project templates already link shots to locations and track shoots through a status pipeline; Storiiboard adopts those habits so the app matches how he already works.

Notion source	What it shows	What Storiiboard adopts
Projects database	Client, type (photo, video, audio), budget, shoot date, due date, payment due, status from Discovery / Contract through Production, Editing, Revising, Pending Invoice, Complete	Same fields in project.json and a status pill on each project card
Brand Collab shot list	Shot name plus location, a deliverables checklist, example content, point of contact	Confirms the core link: every shot carries a location. Add a deliverables checklist and client contact to each project
YouTube video template	Pre-production, post and upload checklists; b-roll color-coded as already have, need to film, or need to source	Shot status on every frame: **Have**, **Need to shoot**, **Source**. The schedule only pulls "Need to shoot" frames into shoot days
5 Free Tools That Fix Claude's Front-End	Playwright for self-testing; design.md files as a design system to hand Claude; a taste skill against generic AI looks	Phase 0 writes docs/design.md from the /prototype look, so Claude Code builds to it. Playwright is already in the plan. The page also fits the creator-tools hub and is a ready content format (comment a keyword, get the list)

Not in Notion: anything on Storiiboard, Bey County, call sheets or locations, so those still come from Kevin directly.

What we take from Notion

Notion's strength is one set of records shown many ways. Storiiboard applies that to film, with the structure already built so nobody starts from a blank page.



data model · one shot table, many views

Edit a shot once and the storyboard, shot list, schedule, call sheet and deck all change with it.

Notion pattern	Storiiboard version	Phase
Every item is a block: id, type, properties, children, parent (Notion's data model)	Every item is a record with the same shape. Changing its type keeps its data, so a reference image becomes a frame without losing anything	0
Edits saved as small operations	Every edit is an operation. Operations drive undo, autosave and the .history log	0
One database, many views	Shots are one table; the storyboard grid, shot list, schedule and call sheet are views of it	1 to 3
Custom properties	Users add their own fields to shots and locations, such as drone, VFX or budget	1

Notion pattern	Storiiboard version	Phase
Slash menu, keyboard-first	"/" inserts a frame, scene, note or location; arrow keys and shortcuts move between frames	1
Relations (how they work)	Each shot links to its scene, location and shoot day. Edit a location once and every shot and call sheet updates	2
Rollups	Each shoot day totals its shots, setups and estimated hours; each location lists its shots	2
Import and export	Import a shot list CSV; export JSON, CSV and PDF	2 to 4
Content shown in several places	Location contacts, parking and safety notes appear on every call sheet that uses that location	3
Templates and a template gallery	Project templates for commercials, music videos, shorts and docs, as plain files people share on GitHub or Patreon	4 to 5

Where Storiiboard goes further

- **Offline:** Notion's offline mode works per device, downloads only the first 50 rows of a database, and embeds don't load offline (Notion help). Storiiboard is offline by design, and embedded videos keep a cached thumbnail so the board still reads on set.
- **No blank page:** Notion makes you build your own system. Storiiboard ships the film structure ready to use, and customization sits on top.
- **Ownership:** Notion keeps your data on its servers. Storiiboard projects are plain files on your drive.
- **Skipped for now:** wikis, team spaces, real-time collaboration and AI agents.

Architecture and stack

One React codebase runs the desktop app (inside Tauri) and the web companion. The desktop app saves to a single .sbd file and works offline; only the web companion needs a server.

Layer	Choice	Why
App UI	Vite, React, TypeScript, Tailwind	Same UI code powers the web companion
Desktop shell	Tauri 2	Installers are a fraction of Electron's size and lighter on memory; uses each OS's own browser engine, so test on Mac and Windows
Desktop, fallback	Electron	Same Chromium everywhere; switch if rendering differs between Mac and Windows
Project file	.sbd, a SQLite database, through Tauri's SQL plugin	One crash-safe file that saves every edit instantly
Canvas	react-konva for sketching, moodboards, the bird's-eye view and deck pages	Free (MIT); handles images, transforms and PNG export
State	Zustand; every edit is an operation	Undo, history, web suggestions and AI edits all share one path
Video	Bundled ffmpeg (LGPL build) makes playback copies with the computer's hardware encoder, grabs stills and trims clips	Mac and Windows play different formats natively; playback copies make every file play smoothly
Weather	Open-Meteo: hourly forecast, sunrise and sunset; no API key	Free for non-commercial apps; donations are allowed, paid features are not
Golden hour	SunCalc library, computed offline	Works on set with no signal
Maps and addresses	OpenStreetMap tiles, plus Nominatim to look up an address the user types	Free; one search per user action, results cached, no autocomplete, app named in every request
Maps, optional	User pastes their own Google Maps key in Settings	Google needs a billing account; each user stays inside its monthly free usage (pricing)

Layer	Choice	Why
Cloud links	Dropbox public folder links download as one zip; Google Drive folders shared with "anyone with the link" are listed and downloaded through the Drive API	No sign-in for public links; private folders need sign-in in a later phase
Web companion	Supabase (Postgres, magic-link sign-in, storage, row-level security), hosted on Vercel	The only server; free tiers cover a small production
AI	Storiiboard's own MCP server, plus optional in-app chat with the user's own key for Anthropic, OpenAI, Gemini or OpenRouter	Any MCP-capable AI gets full project context; Storiiboard pays nothing for AI
Export	Konva renders pages to PNG, pdf-lib builds the PDF	Pixel-matched call sheets, boards and decks
Quality	Vitest, Playwright, GitHub Actions building Mac and Windows installers	Claude Code can check its own work on every PR

The .sbd project file

Every Storiiboard project is one .sbd file. Double-clicking it opens the app, like a .prproj or .drp.

- Inside, a .sbd is a SQLite database, the same format behind Lightroom catalogs and most phone apps. It saves each edit the moment it happens and survives crashes and power cuts.
- It holds all project data, the edit history, images and thumbnails. Video stays outside as linked files or cloud links with a cached thumbnail, so the .sbd stays small enough to email or drop in a shared folder.
- Collect Project packs the .sbd and every linked file into one .sbdpack zip for handoff.
- The format stays open: the schema is documented in the repo, and every project exports to JSON and CSV.
- Each file records its schema version; older projects are backed up, then upgraded when opened.

This replaces the earlier folder-of-JSON plan. The trade-off is that a project is no longer easy to compare line by line in git; the JSON export covers that.

AI assistant (MCP)

Storiiboard runs its own MCP server, so Claude or any MCP-capable AI can read the whole project and propose changes. Nothing changes until a person approves it, and the app works fully without AI (MCP transports).

- **Read:** scenes, shots, schedule, call sheets, cast and crew, gear, locations, bird's-eye plans, suggestions.
- **Propose:** add or edit shots, reorder the schedule, draft a call sheet, fill a gear list. Every change lands in the approval queue.
- **Check:** scan the project for the mistakes that cause miscommunication on set:
  - a cast member is called on a day they marked unavailable
  - a shot needs gear that is not on that day's list
  - a golden-hour shot is scheduled at midday, or an exterior lands on a day with rain forecast
  - a location has no address, contact or parking note
  - a department is missing from a call sheet that needs it, such as sound on dialogue scenes
  - hair and makeup time is too short before a cast call
  - the storyboard lens differs from the camera lens in the bird's-eye plan
- **Script to scenes:** read a script and propose the full scene list, locations and cast roles, then a breakdown of props, wardrobe, vehicles, stunts and VFX per scene.
- **Blender:** working alongside a Blender MCP server, turn a bird's-eye plan into a 3D scene and adjust it on request ("move the key light to camera left").

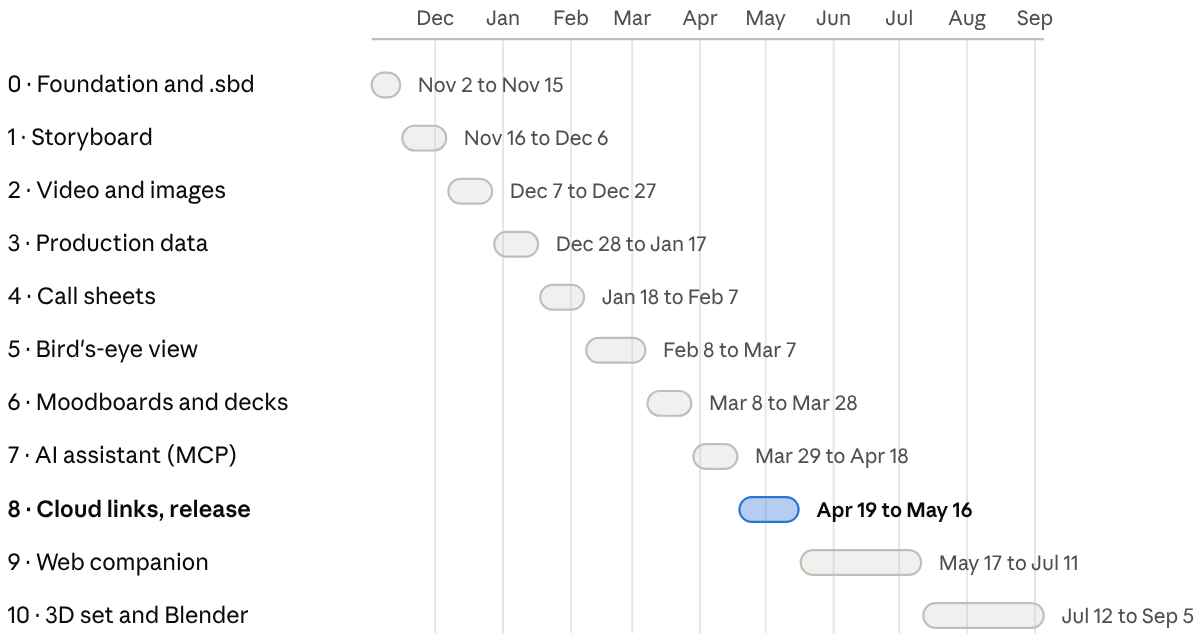
For the content series, the line stays honest: AI works for the crew and catches mistakes; it never makes the art.

Roadmap

About 36 weeks of part-time building takes the desktop app to a public release in mid-May 2027, with the web companion live by July. Each phase is usable on its own, so the build can pause at any gate without losing work.

The desktop app goes public in May; the web companion follows by July

Estimated part-time schedule from a Nov 2, 2026 start



roadmap · 6 phases, 1 gate each

Dates are estimates from a Nov 2 start. A phase moves on only when its gate passes on Kevin's own material, not demo data.

Phase	Delivers	Gate
0 · Foundation and .sbd	Desktop shell, .sbd file with autosave and history, installers from CI, prototype archived	A project survives a force-quit, and double-clicking a .sbd opens it
1 · Storyboard	Script import into scene shells, frames, shot specs, sketching, "/" menu, custom fields	A real scene is boarded in one sitting
2 · Video and images	Playback copies, frame stepping, in and out, crop, still and clip capture, attachments	A still from a reference clip lands in a frame in a few clicks
3 · Production data	Locations, cast and crew, departments, gear lists, shoot days	A real shoot's people, places and gear are entered

Phase	Delivers	Gate
4 · Call sheets	Call sheet per day with weather, sun times, map and hospital; PDF	Used on a real shoot day
5 · Bird′s-eye view	Top-down plans with walls, cast, cameras with true field of view, lights, department layers	A gaffer plans a real setup with it
6 · Moodboards and decks	Department moodboards, pitch decks from frames	A deck built in under an hour
7 · AI assistant	MCP server, mistake checks, approval queue	The checks catch a real scheduling or gear mistake
8 · Cloud links and release	Dropbox and Google Drive links, Collect Project, signed installers, public repo, Patreon	A stranger installs it and opens a shared project
9 · Web companion	Call sheets, compressed storyboard, bird′s-eye viewer, availability, comments and suggestions, company workspaces	Cast and crew get their call sheet from the site and send a suggestion

Phase 10, 3D set and Blender (July to September 2027): 3D set preview, glTF export, Open in Blender, the simple Blender layout. Gate: a DP checks a real setup through the 3D camera before the shoot. After that: device-wide search for missing files, sign-in for private cloud folders, real-time collaboration, Resolve and Premiere marker export.

Claude Code hand-off

Claude Code builds one phase at a time on a v2 branch, one pull request per milestone; I act as PM, checking each PR against the phase′s exit criteria before Kevin merges.

Repo setup

1. Create a v2 branch. Move the current files into /prototype as read-only design reference.
2. Scaffold the app at the repo root. Add docs/PLAN.md (this doc′s scope, stack and roadmap) and CLAUDE.md (below).

3. Turn on GitHub Actions so every PR builds a Mac and a Windows installer before any feature work.
4. Keep the repo private until Phase 8, then make it public with a license, README and contributing guide.

Working rhythm

- Start every phase in plan mode; Kevin approves the plan, then Claude Code builds.
- One PR per milestone, small enough to test in 15 minutes from its installer build.
- CI must be green; every PR adds or updates a test for the flow it touches.
- Claude Code updates docs/PLAN.md status at the end of each PR.

CLAUDE.md

165 lines, bullets only, so Claude Code can scan it fast. Save it as CLAUDE.md at the repo root.

```
# Storiiboard — CLAUDE.md

## What this is
- Open-source pre-production tool for filmmakers, producers and content
creators.
- Mission: production tools people own, instead of renting them from
corporations.
- Core: fast storyboarding. Every shot links to its scene, location, shoot
day, people and gear.
- Production flows out of the storyboard: schedule, call sheets, gear lists,
bird's-eye plans.
- Three surfaces:
  - Desktop app (Tauri): where production decisions are made; works offline.
  - Web companion: cast, crew and clients view, comment, suggest, mark
availability.
  - MCP server: AI assistants read the project and propose changes.
- First user is Kevin Matos, a working DP. Build for real shoots, not demos.

## Read first
- docs/PLAN.md: current phase, scope and exit criteria.
- Build only the current phase. Ask before starting anything outside it.
- /prototype is visual reference only. Match its look; never import from it.

## Film vocabulary (use it correctly)
- Shot sizes: EWS, WS, MS, MCU, CU, ECU, insert.
- Lens focal length in mm; sensor width in mm; aperture as T-stop.
```

```
- Scene headings: INT/EXT, DAY/NIGHT, scene number; shot numbers 1A, 1B, 1C.
- Setup = one camera position and lighting state; several shots can share
one.
- Call time, crew call, shooting call, wrap, company move.
- Departments: direction, camera, grip and electric, art, HMU, wardrobe,
sound, production, post, cast, client.
- Editors need scene and shot numbers that match the slate exactly.

## Stack
- Shell: Tauri 2 (Rust). UI: React, TypeScript (strict), Vite, Tailwind.
- State: Zustand. Every canvas: react-konva.
- Project file: SQLite through tauri-plugin-sql.
- Video: ffmpeg sidecar, LGPL build, for proxies, stills and trims.
- PDFs: pdf-lib. Sun times: SunCalc. 3D (Phase 10): three.js through react-
three-fiber.
- Web companion: same React code, Supabase backend, Vercel hosting.
- Tests: Vitest (unit), Playwright (end to end).
- CI: GitHub Actions builds Mac and Windows installers on every PR.

## Repo layout
- src/app: screens and routing.
- src/features/<tool>: storyboard, video, birdseye, locations, people, gear,
schedule, callsheets, moodboards, decks, cloud, availability, suggestions.
- src/core/model: record types and schema.
- src/core/ops: operations, undo, history.
- src/core/storage: .sbd read and write. The only code that touches the disk.
- src/core/net: weather, geocoding, maps, cloud links.
- src/core/checks: consistency checks shared by the app and MCP.
- src/ui: shared components.
- src-tauri: Rust shell, .sbd file association, ffmpeg sidecar.
- mcp: MCP server.
- web: web companion entry point.
- docs: PLAN.md, SBD_FORMAT.md, MCP.md.

## The .sbd project file
- One project = one .sbd file = one SQLite database.
- It holds all records, the edit log, images and thumbnails.
- Video stays outside the file: a linked path or cloud link, plus a cached
thumbnail.
- Store media paths relative to the .sbd when possible; absolute otherwise.
- Every table has id (UUID v4), created_at, updated_at.
- The meta table holds schema_version. Migrations live in
src/core/storage/migrations.
```

```
- Copy the file to a backup before running any migration.
- .sbdpack = zip of the .sbd plus all linked media ("Collect project").
- Document every schema change in docs/SBD_FORMAT.md.
- JSON and CSV export must always round-trip without data loss.

## Data model
- Every record has: id, type, props, parent_id, ordered children.
- Changing a record's type never drops its props.
- Records: project, scene, shot, setup, media, location, person, department,
gear_item, shoot_day, callsheet, plan, plan_item, board, board_item, design,
comment, suggestion, availability.
- A shot links to: one scene, at most one location, shoot_day and setup, and
many people, gear items and media.
- Views (storyboard, shot list, schedule, call sheet, deck) read records.
They never copy data.
- Rollups (day totals, a location's shot list) are computed, never stored.
- Users can add custom props to shots, locations and people.

## Edits, undo and autosave
- Every change is an operation: id, record_id, kind, before, after, author,
timestamp.
- The UI, MCP, imports and web suggestions all apply edits through
src/core/ops.
- Undo and redo walk the op log. Every canvas action is undoable.
- Each op commits to SQLite in a transaction right away. There is no Save
button.
- Users can name versions; automatic snapshots are pruned after 30 days.

## Tool rules
- Storyboard: keyboard-first; "/" menu inserts frame, scene, note, location;
drag to reorder; auto shot numbers.
- Video: play proxies, never originals. Comma and period step frames; I and O
set in and out.
  - Crop, then grab a still or a short clip straight into a frame.
  - Proxies are H.264 from the hardware encoder (VideoToolbox on Mac, Media
Foundation on Windows).
- Bird's-eye view: top-down plan in real units (meters or feet, a setting).
  - Items: walls, doors, windows, set pieces, cast marks and paths, cameras,
lights, flags, diffusion.
  - Camera field of view = 2 * atan(sensor_width / (2 * focal_length)).
  - Layers per department: camera (DP), lighting (gaffer), grip, art, cast
blocking.
  - One plan per scene setup, linked to its shots.
```

```
- Call sheets: generated per shoot_day from linked records, with editable
notes; PDF and web.
  - Include call times, scenes, cast with character and HMU times, crew by
department, locations with map and parking,
    nearest hospital, weather, sunrise and sunset, golden hour, gear and
notes.
- Moodboards: one per department; they link to scenes, characters and
locations.
- Cloud links: one shared link per project; list, download and relink media.
- Missing media: flag it clearly and offer relink. Device-wide file search is
a later phase.
- Script import: Fountain, Final Draft (.fdx) and PDF.
  - Parse scene headings by rule, not AI; create scene, location and
character records.
  - AI breakdown (props, wardrobe, vehicles, stunts, VFX) arrives only as
suggestions.
- 3D set (Phase 10): react-three-fiber blockout built from the bird's-eye
plan; look through any camera.
  - Export glTF (.glb) with cameras and KHR_lights_punctual lights.
  - Open in Blender: run the user's installed Blender in the background with
a bundled Python script.
  - Never bundle Blender. Ship a Storiiboard application template with a
simplified layout.

## Network rules
- Weather: Open-Meteo, no key, non-commercial use only; credit it on call
sheets (CC BY 4.0).
- Geocoding: Nominatim only when the user presses Search; at most 1 request
per second;
  app name in the User-Agent; cache results in the .sbd; never autocomplete.
- Google Maps only with the user's own key from Settings.
- Dropbox: a public folder link with dl=1 downloads as one zip.
- Google Drive: "anyone with the link" folders through the Drive API with a
project API key.
- Cache every lookup in the .sbd with a fetched_at timestamp.
- Everything except lookups, cloud links and publishing works offline.

## Web companion
- Read-only views: call sheets, compressed storyboard (images only, no video,
no tools), latest bird's-eye plans.
- Interactive: comments, suggestions, availability (whole dates only, no
hours).
- The desktop app publishes a snapshot. The web never edits the .sbd
```

```
directly.
- Suggestions and comments sync back to the desktop approval queue.
- Access by per-person invite link (magic link). No public projects.
- Contact details are visible only to people on that call sheet.
- Supabase row-level security on every table, with tests for each policy.
- Company workspaces: roles owner, approver (CEO, marketing), creator, crew,
editor, cast, client.
- Published snapshots expire 30 days after the last shoot day by default.

## MCP server
- Transports: stdio for Claude Desktop and Claude Code; local HTTP bound to
127.0.0.1 with an Origin check and a token.
- Read tools: project summary, scenes, shots, schedule, call sheets, people,
gear, locations, plans, suggestions.
- Write tools only create suggestions. A person approves them in the app.
- The check tool runs src/core/checks and returns findings.
- The script tool reads an imported script and proposes scenes and a
breakdown as suggestions.
- Checks:
  - Cast called on a day they marked unavailable.
  - A shot needs gear that is not on that day's list.
  - Golden-hour shot scheduled at midday; exterior on a day with rain
forecast.
  - Location missing address, contact or parking note.
  - Department missing from a call sheet that needs it (for example, sound on
dialogue scenes).
  - HMU time too short before a cast member's call.
  - Storyboard lens differs from the camera lens in the bird's-eye plan.
- Never send project data anywhere except the client that called the server.
- Optional in-app chat uses the user's own key (Anthropic, OpenAI, Gemini,
OpenRouter) from the OS keychain.

## Security and privacy
- No secrets in code or the repo. User keys live in the OS keychain.
- Call sheets hold phone numbers and addresses: treat them as private data.
- Never log personal data.

## Code style
- TypeScript strict; no `any`.
- Small components, one feature per folder.
- Name things with the film terms users already know.
- Comment the why, not the what. People will fork this code.
- Accessible: everything keyboard reachable, labels on icon buttons, AA
```

```
contrast.

## Design
- Dark charcoal base; pastel accents: dusty rose, sage, lavender, warm sand.
- DM Sans for UI, Playfair Display for display type.
- Dense, quiet UI like Linear. The storyboard images supply the color.

## Workflow
- Start each phase in plan mode and wait for Kevin's approval.
- One PR per milestone, small enough to review in 15 minutes.
- Every PR adds or updates tests for the flow it touches.
- Before saying done: `npm run lint && npm test && npm run e2e`.
- Ask before adding a dependency or changing the .sbd schema or the MCP
tools.
- Update the status in docs/PLAN.md at the end of each PR.
```

Phase 0 kickoff prompt

```
Read CLAUDE.md and docs/PLAN.md. We are on Phase 0: Foundation and .sbd.
In plan mode, propose how you will:
1. Move the existing prototype into /prototype and remove its
   client-side password gate.
2. Scaffold Tauri 2 + Vite + React + TypeScript + Tailwind at the root,
   using the repo layout in CLAUDE.md.
3. Build src/core/storage and src/core/ops: create, open and autosave a
   .sbd file (SQLite), with an operation log, undo and redo, and a
   meta table holding schema_version.
4. Register .sbd with Mac and Windows so double-clicking opens the app.
5. Write docs/SBD_FORMAT.md for the first schema.
6. Add Vitest, Playwright, ESLint and a GitHub Actions workflow that
   builds Mac and Windows installers.
7. Ship a Projects screen (New, Open, Recent) that matches the
   prototype's look.
Exit criteria: CI green on both platforms; a project survives
force-quitting mid-edit; double-clicking a .sbd opens it; undo works
across an app restart.
```

Prompt template for later phases

```
Read CLAUDE.md and docs/PLAN.md. Phase [N]: [name].
Goal: [one sentence]. Exit criteria: [list from PLAN.md].
Reference screens in /prototype: [files].
Plan first, then build in PRs of one milestone each.
```

Decisions, risks and next steps

Audience is settled: Kevin first, then a free open-source desktop app for filmmakers, producers and creators, with a web companion for everyone on set. Three calls unblock Phase 0; the rest can wait until Phase 8.

Decisions needed

Decision	Options	Recommendation	Needed by
Desktop shell	Tauri (small, uses each OS's browser engine) or Electron (bigger, identical everywhere)	Tauri; switch only if the canvas or video misbehaves on one OS	Phase 0
Repo	v2 branch in the same repo, or a fresh repo	v2 branch, prototype kept in /prototype	Phase 0
Inside the .sbd	One SQLite database, or a zipped folder of JSON files	SQLite: crash-safe and fast; JSON export keeps it open	Phase 0
License	MIT (anyone can do anything, including closed forks) or GPL (forks must stay open)	MIT for maximum reach; GPL if improvements should flow back	Phase 8
Patreon model	Donations only, or paid tiers that unlock features	Donations only: perks like early builds, templates and a Discord, never locked features. Paid features would break Open-Meteo's free terms	Phase 8
Code signing	Pay for Apple and Windows signing, or ship unsigned	Sign the Mac build at least; unsigned apps show scary warnings	Phase 8
Web companion hosting	Kevin hosts one site for everyone, or each production hosts its own	Host it for Kevin's productions first with a self-host guide; decide on an open hosted service once costs are known	Phase 9

Risks

Risk	Fallback
Ten phases is a long build; it may never feel finished	The desktop app goes public at Phase 8 even if the web companion slips
Camera RAW formats such as BRAW and R3D need maker SDKs that ffmpeg doesn't include	Accept common formats and ProRes first; RAW users import a reference export; maker SDKs later
Rendering or video playback differs between Mac and Windows under Tauri	Test both in CI from Phase 0; Electron is the escape hatch
The web companion leaks phone numbers or addresses	Private invite links, contact details only for people on that sheet, row-level security with tests, snapshots that expire
Free location services throttle or change terms	Cache every lookup; the app keeps working without them; Google key as backup
Google Drive links break or hit quota	Support Dropbox first; show a clear error and let users relink files by hand
AI proposes a wrong change	AI can only suggest; a person approves everything
Big image boards get slow	Thumbnails on the canvas, originals only on export
Claude Code drifts or breaks earlier work	CLAUDE.md rules, CI tests, one milestone per PR
Open-source support load after release	Issue templates, a clear README, a Patreon tier for priority requests
Limited build hours alongside client work	Each phase is usable on its own

Next steps

- [ ] Kevin confirms Tauri and the v2 branch
- [ ] Remove the client-side password and change it anywhere it is reused
- [ ] Create the v2 branch and add CLAUDE.md and docs/PLAN.md
- [ ] Run the Phase 0 kickoff prompt in Claude Code
- [ ] Pick the pilot shoot days for the call sheet phase

- [ ] Before Phase 8: choose the license, Patreon tiers and signing

Sources

- Open-Meteo terms
- Nominatim usage policy
- Google Maps Platform pricing
- tldraw pricing
- Electron vs Tauri, 2026
  - The data model behind Notion
  - MCP transports specification